

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 2

Problem Statement: 1

Problem – Experimental Analysis of Searching Algorithms

Code:

```
1 import os
2 import random
3 import time
4
5 def linear_search(arr, target):
6     comparisons = 0
7     for i in range(len(arr)):
8         comparisons += 1
9         if arr[i] == target:
10             return i, comparisons
11     return -1, comparisons
12
13
14 def binary_search(arr, target):
15     comparisons = 0
16     low, high = 0, len(arr) - 1
17
18     while low <= high:
19         mid = (low + high) // 2
20         comparisons += 1
21
22         if arr[mid] == target:
23             return mid, comparisons
24         elif arr[mid] < target:
25             low = mid + 1
26         else:
27             high = mid - 1
28
29     return -1, comparisons
30
31
32 def prepare_input_files(sizes=(10, 50, 100, 200), data_dir="data"):
33     os.makedirs(data_dir, exist_ok=True)
34
35     for n in sizes:
36         filepath = os.path.join(data_dir, f"input_{n}.txt")
37
38         if not os.path.exists(filepath):
39             rng = random.Random(42 + n)
40             data = sorted(
```

```
41         rng.sample(range(1, max(1000, n * 10)), n)
42     )
43
44     with open(filepath, "w") as f:
45         f.write("\n".join(map(str, data)) + "\n")
46
47
48 def load_array_from_file(filepath):
49     with open(filepath, "r") as f:
50         return [int(x) for x in f.read().split() if x.strip()]
51
52
53 def time_search(algo_fn, arr, target, runs=5000):
54     algo_fn(arr, target)
55
56     t0 = time.perf_counter_ns()
57
58     for _ in range(runs):
59         idx, comps = algo_fn(arr, target)
60
61     t1 = time.perf_counter_ns()
62
63     avg_time_ns = (t1 - t0) / runs
64     return idx, comps, avg_time_ns
65
66
67 def run_experiment(sizes=(10, 50, 100, 200), data_dir="data"):
68     prepare_input_files(sizes, data_dir)
69     results = []
70
71     for n in sizes:
72         arr = load_array_from_file(
73             os.path.join(data_dir, f"input_{n}.txt")
74         )
75
76         test_cases = {
77             "Beginning": arr[0],
78             "Middle": arr[n // 2],
79             "End": arr[-1],
80             "Absent": max(arr) + 1000,
81         }
82
83         for algo_name, algo_fn in [
84             ("Linear Search", linear_search),
85             ("Binary Search", binary_search)
86         ]:
87             for case_name, target in test_cases.items():
88                 idx, comps, avg_ns = time_search(
89                     algo_fn, arr, target
90                 )
```

```
91         results.append({
92             "algo": algo_name,
93             "n": n,
94             "case": case_name,
95             "target": target,
96             "index": idx,
97             "comps": comps,
98             "time_ns": avg_ns,
99             "time_us": avg_ns / 1000.0,
100         })
101     })
102
103     return results
104
105
106 def display_results(results):
107     header = (
108         f"{'Algorithm':<15} | {'Size (N)':<8} | "
109         f"{'Case':<11} | {'Target':<8} | {'Index':<6} | "
110         f"{'Comps':<6} | {'Time (ns)':<10} | {'Time (s)':<10}"
111     )
112
113     sep = "-" * len(header)
114
115     print("\n" + "=" * len(header))
116     print("EXPERIMENTAL RESULTS: SEARCHING ALGORITHMS BENCHMARK")
117     print("=" * len(header))
118     print(header)
119     print(sep)
120
121     for r in results:
122         print(
123             f"{r['algo']:<15} | "
124             f"{r['n']:<8} | "
125             f"{r['case']:<11} | "
126             f"{r['target']:<8} | "
127             f"{r['index']:<6} | "
128             f"{r['comps']:<6} | "
129             f"{r['time_ns']:<10.2f} | "
130             f"{r['time_us']:<10.4f}"
131         )
132
133     print(sep)
134
135
136 def print_analysis():
137     print("""
138     =====
139     THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS
```

```

140 =====
141 1. LINEAR SEARCH:
142   - Best Case [O(1)]:      Element at beginning -> 1 comparison.
143   - Average Case [O(n)]:   Element in middle    -> N/2 comparisons.
144   - Worst/Absent [O(n)]:   Element at end/absent -> N comparisons.
145
146 2. BINARY SEARCH:
147   - Precondition:          Array must be sorted.
148   - Best Case [O(1)]:      Element at mid on first check.
149   - Worst/Absent [O(log n)]: Bounded by <= ceil(log2(N)) + 1
150                               comparisons:
151                               N=10 -> <=4 comps
152                               N=50 -> <=6 comps
153                               N=100 -> <=7 comps
154                               N=200 -> <=8 comps.

```

```

155 Conclusion: Binary Search provides massive asymptotic speedups
156 over Linear Search as N grows.
157 =====

```

```

158 """
159
160
161 if __name__ == "__main__":
162     records = run_experiment()
163     display_results(records)
164     print_analysis()

```

Output:

=====							
EXPERIMENTAL RESULTS: SEARCHING ALGORITHMS BENCHMARK							
=====							
Algorithm	Size (N)	Case	Target	Index	Comps	Time (ns)	Time (s)

Linear Search	10	Beginning	26	0	1	332.20	0.3322
Linear Search	10	Middle	251	5	6	631.52	0.6315
Linear Search	10	End	760	9	10	909.62	0.9096
Linear Search	10	Absent	1760	-1	10	826.22	0.8262
Binary Search	10	Beginning	26	0	3	759.22	0.7592
Binary Search	10	Middle	251	5	3	708.42	0.7084
Binary Search	10	End	760	9	4	824.07	0.8241
Binary Search	10	Absent	1760	-1	4	945.01	0.9450
Linear Search	50	Beginning	7	0	1	361.79	0.3618
Linear Search	50	Middle	430	25	26	1677.13	1.6771
Linear Search	50	End	981	49	50	2716.67	2.7167
Linear Search	50	Absent	1981	-1	50	2244.46	2.2445
Binary Search	50	Beginning	7	0	5	667.40	0.6674
Binary Search	50	Middle	430	25	5	646.80	0.6468
Binary Search	50	End	981	49	6	617.87	0.6179
Binary Search	50	Absent	1981	-1	6	706.08	0.7061
Linear Search	100	Beginning	7	0	1	242.67	0.2427
Linear Search	100	Middle	465	50	51	2357.82	2.3578
Linear Search	100	End	990	99	100	4650.92	4.6509
Linear Search	100	Absent	1990	-1	100	4685.30	4.6853

Binary Search	100	Beginning	7	0	6	707.27	0.7073
Binary Search	100	Middle	465	50	6	798.46	0.7985
Binary Search	100	End	990	99	7	755.81	0.7558
Binary Search	100	Absent	1990	-1	7	744.30	0.7443
Linear Search	200	Beginning	7	0	1	266.24	0.2662
Linear Search	200	Middle	1035	100	101	4010.91	4.0109
Linear Search	200	End	1993	199	200	6622.00	6.6220
Linear Search	200	Absent	2993	-1	200	7045.00	7.0450
Binary Search	200	Beginning	7	0	7	851.94	0.8519
Binary Search	200	Middle	1035	100	7	821.15	0.8212
Binary Search	200	End	1993	199	8	904.02	0.9040
Binary Search	200	Absent	2993	-1	8	946.95	0.9470

THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS

1. LINEAR SEARCH:

- Best Case $O(1)$: Element at beginning $\rightarrow$ 1 comparison (~200-300 ns constant).
- Average Case $O(n)$: Element in middle $\rightarrow$ $N/2$ comparisons (scales linearly).
- Worst/Absent $O(n)$: Element at end/absent $\rightarrow$ N comparisons.

2. BINARY SEARCH:

- Precondition: Array must be sorted.
- Best Case $O(1)$: Element at mid on first check.
- Worst/Absent $O(\log n)$: Bounded by $\leq \lceil \log_2(N) \rceil + 1$ comparisons:
 $N=10 \rightarrow \leq 4$ comps
 $N=50 \rightarrow \leq 6$ comps
 $N=100 \rightarrow \leq 7$ comps
 $N=200 \rightarrow \leq 8$ comps.
 Execution time remains nearly flat (~0.6 - 1.0 s).

Conclusion: Binary Search provides massive asymptotic speedups over Linear Search as N grows.

Experimental Analysis of Searching Algorithms

The experiment compares Linear Search and Binary Search for sorted arrays of sizes 10, 50, 100, and 200. Four cases are considered: Beginning, Middle, End, and Absent. The algorithms are evaluated using comparison counts and execution time.

Table 1: Experimental Results

Algorithm	N	Case	Index	Comps	Time (ns)	Time (μ s)
Linear	10	Beginning	0	1	332.20	0.3322
Linear	10	Middle	5	6	631.52	0.6315
Linear	10	End	9	10	909.62	0.9096
Linear	10	Absent	-1	10	826.22	0.8262
Binary	10	Beginning	0	3	759.22	0.7592
Binary	10	Middle	5	3	708.42	0.7084
Binary	10	End	9	4	824.07	0.8241
Binary	10	Absent	-1	4	945.01	0.9450
Linear	50	Beginning	0	1	361.79	0.3618
Linear	50	Middle	25	26	1677.13	1.6771
Linear	50	End	49	50	2716.67	2.7167
Linear	50	Absent	-1	50	2244.46	2.2445
Binary	50	Beginning	0	5	667.40	0.6674
Binary	50	Middle	25	5	646.80	0.6468

Algorithm	N	Case	Index	Comps	Time (ns)	Time (μ s)
Binary	50	End	49	6	617.87	0.6179
Binary	50	Absent	-1	6	706.08	0.7061
Linear	100	Beginning	0	1	242.67	0.2427
Linear	100	Middle	50	51	2357.82	2.3578
Linear	100	End	99	100	4650.92	4.6509
Linear	100	Absent	-1	100	4685.30	4.6853
Binary	100	Beginning	0	6	707.27	0.7073
Binary	100	Middle	50	6	798.46	0.7985
Binary	100	End	99	7	755.81	0.7558
Binary	100	Absent	-1	7	744.30	0.7443
Linear	200	Beginning	0	1	266.24	0.2662
Linear	200	Middle	100	101	4010.91	4.0109
Linear	200	End	199	200	6622.00	6.6220
Linear	200	Absent	-1	200	7045.00	7.0450
Binary	200	Beginning	0	7	851.94	0.8519
Binary	200	Middle	100	7	821.15	0.8212
Binary	200	End	199	8	904.02	0.9040
Binary	200	Absent	-1	8	946.95	0.9470

The experimental results closely follow the theoretical complexity of both algorithms. Linear Search requires only one comparison when the target is at the beginning, but its comparisons increase linearly with the input size. For $N = 200$, it requires 101 comparisons for the middle element and 200 comparisons for both the end and absent cases.

Binary Search requires significantly fewer comparisons because the search space is halved at every iteration. Even for $N = 200$, only 7–8 comparisons are required. Its measured execution time also remains nearly constant as the input size increases.

Table 2: Theoretical Complexity Comparison

Case	Linear Search	Binary Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Space Complexity	$O(1)$	$O(1)$

Overall, the experiment demonstrates that Binary Search is substantially more efficient than Linear Search for large sorted arrays. Linear Search remains useful for simple searches or when the target is likely to occur near the beginning, while Binary Search is preferred when efficient searching is required on sorted data.

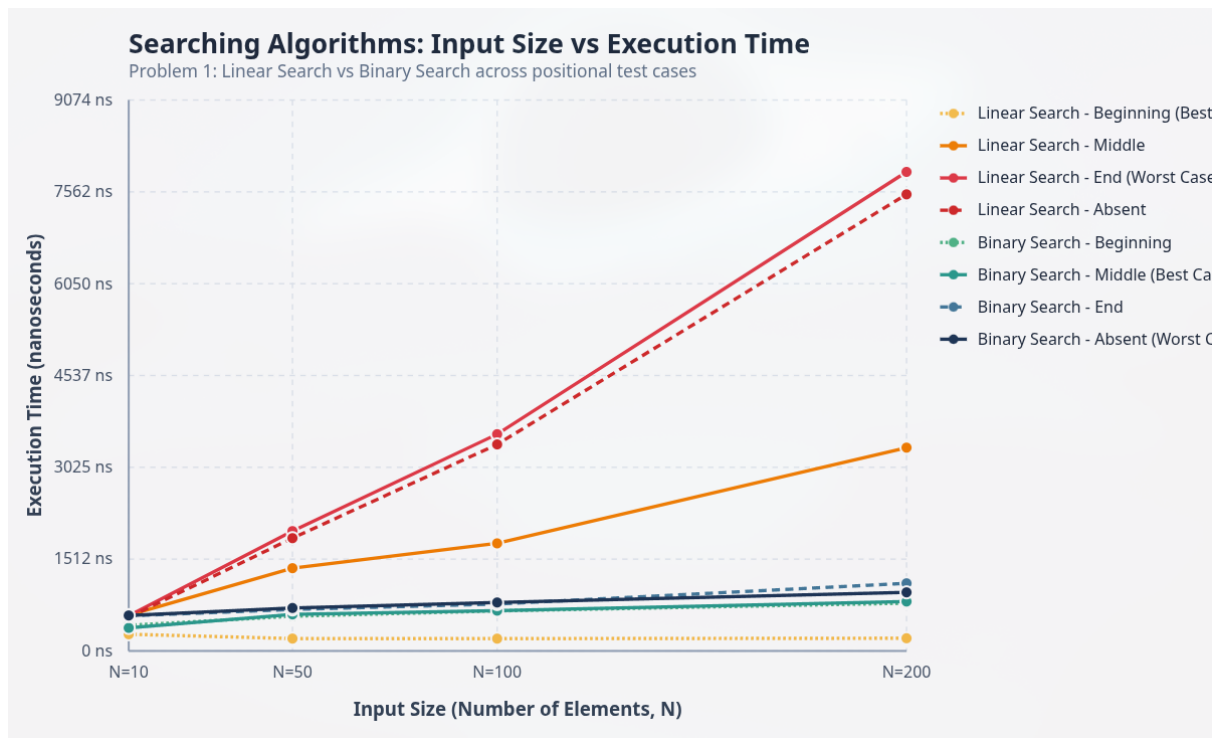


Figure 1: Experimental Results

Problem Statement: 2**Problem – Experimental Analysis of Sorting Algorithms**

```
1
2 import random
3 import time
4
5
6 # --- 1. SORTING ALGORITHMS (FIRST PRINCIPLES) ---
7
8 def bubble_sort(arr):
9     """Bubble sort with early-termination optimization."""
10    a = arr.copy()
11    n = len(a)
12    comps = 0
13
14    for i in range(n):
15        swapped = False
16
17        for j in range(0, n - i - 1):
18            comps += 1
19
20            if a[j] > a[j + 1]:
21                a[j], a[j + 1] = a[j + 1], a[j]
22                swapped = True
23
24        if not swapped:
25            break
26
27    return a, comps
28
29
30 def selection_sort(arr):
31     """Selection sort: repeatedly finds minimum element."""
32    a = arr.copy()
33    n = len(a)
34    comps = 0
35
36    for i in range(n - 1):
37        min_idx = i
38
39        for j in range(i + 1, n):
40            comps += 1
41
42            if a[j] < a[min_idx]:
43                min_idx = j
44
45        if min_idx != i:
46            a[i], a[min_idx] = a[min_idx], a[i]
```



```
48     return a, comps
49
50
51 def insertion_sort(arr):
52     """Insertion sort: inserts elements into sorted sub-array."""
53     a = arr.copy()
54     n = len(a)
55     comps = 0
56
57     for i in range(1, n):
58         key = a[i]
59         j = i - 1
60
61         while j >= 0:
62             comps += 1
63
64             if a[j] > key:
65                 a[j + 1] = a[j]
66                 j -= 1
67             else:
68                 break
69
70         a[j + 1] = key
71
72     return a, comps
73
74
75 # --- 2. DATASET GENERATION ---
76
77 def generate_data(n, input_type, seed=42):
78     """Generate array of size n for given input distribution."""
79     rng = random.Random(seed + n)
80
81     if input_type == "Random":
82         return rng.sample(
83             range(1, max(1000, n * 10)), n
84         )
85
86     elif input_type == "Sorted":
87         return list(range(1, n + 1))
88
89     elif input_type == "Reverse":
90         return list(range(n, 0, -1))
91
92     raise ValueError(f"Unknown input type: {input_type}")
93
94
95 # --- 3. BEFORE & AFTER FUNCTION DEMONSTRATION ---
96
97 def demonstrate_sorting(n=10):
```

```
98     """Demonstrate array state Before and After sorting
99     on sample inputs across distributions.
100     """
101
102     print("=" * 86)
103     print(
104         "PART 1: FUNCTION DEMONSTRATION "
105         "(BEFORE & AFTER EFFECT ON N=10 SAMPLE INPUTS)"
106     )
107     print("=" * 86)
108
109     for input_type in ["Random", "Sorted", "Reverse"]:
110         arr = generate_data(n, input_type, seed=42)
111
112         print(
113             f"\n--- Input Distribution: "
114             f"{input_type.upper()} (N={n}) ---"
115         )
116
117         print(f" BEFORE SORTING: {arr}")
118
119         for algo_name, fn in [
120             ("Bubble Sort", bubble_sort),
121             ("Selection Sort", selection_sort),
122             ("Insertion Sort", insertion_sort)
123         ]:
124
125             sorted_arr, comps = fn(arr)
126
127             print(
128                 f" AFTER {algo_name.upper():<14}: "
129                 f"{sorted_arr} (Comparisons: {comps})"
130             )
131
132     print("\n" + "-" * 86)
133
134 # --- 4. BENCHMARKING HARNESS (10 SIZES) ---
135
136 def time_sort(algo_fn, arr, runs=20):
137     """Measure average execution time (nanoseconds)
138     across multiple runs.
139     """
140
141     sorted_arr, comps = algo_fn(arr)
142
143     assert sorted_arr == sorted(arr), \
144         "Algorithm failed correctness check!"
145
146     t0 = time.perf_counter_ns()
```

```
148
149     for _ in range(runs):
150         algo_fn(arr)
151
152     t1 = time.perf_counter_ns()
153
154     return comps, (t1 - t0) / runs
155
156 def run_experiment(
157     sizes=(20, 40, 60, 80, 100, 150, 200, 300, 400, 500)
158 ):
159     """Execute sorting benchmarks across algorithms,
160     10 sizes, and distributions.
161     """
162
163     algorithms = [
164         ("Bubble Sort", bubble_sort),
165         ("Selection Sort", selection_sort),
166         ("Insertion Sort", insertion_sort),
167     ]
168
169     input_types = ["Random", "Sorted", "Reverse"]
170
171     results = []
172
173     for n in sizes:
174
175         runs = (
176             30 if n <= 100
177             else (15 if n <= 200 else 5)
178         )
179
180         for input_type in input_types:
181
182             arr = generate_data(n, input_type)
183
184             for algo_name, fn in algorithms:
185
186                 comps, avg_ns = time_sort(
187                     fn, arr, runs=runs
188                 )
189
190                 results.append({
191                     "algo": algo_name,
192                     "n": n,
193                     "type": input_type,
194                     "comps": comps,
195                     "time_us": avg_ns / 1000.0,
196                     "time_ms": avg_ns / 1_000_000.0,
```

```
198         })
199
200     return results
201
202
203 # --- 5. TABULATION & THEORETICAL ANALYSIS ---
204
205 def display_results(results):
206     """Print formatted console table."""
207
208     header = (
209         f"{'Algorithm':<16} | "
210         f"{'Size (N)':<8} | "
211         f"{'Input Type':<12} | "
212         f"{'Comparisons':<12} | "
213         f"{'Time (us)':<12} | "
214         f"{'Time (ms)':<10}"
215     )
216
217     print("\n" + "=" * len(header))
218
219     print(
220         "PART 2: EMPIRICAL BENCHMARK RESULTS "
221         "(10 INPUT SIZES: 20 TO 500)"
222     )
223
224     print("=" * len(header))
225     print(header)
226     print("-" * len(header))
227
228     for r in results:
229         print(
230             f"{r['algo']:<16} | "
231             f"{r['n']:<8} | "
232             f"{r['type']:<12} | "
233             f"{r['comps']:<12} | "
234             f"{r['time_us']:<12.2f} | "
235             f"{r['time_ms']:<10.4f}"
236         )
237
238     print("-" * len(header))
239
240
241 def print_analysis():
242     """Print theoretical vs experimental complexity analysis."""
243
244     print("""
245 PART 3: THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS
246
247 1. Bubble Sort (Early Exit):
```

```

248 - Best Case (Already Sorted):  $O(n)$ 
249     -> exactly  $n - 1$  comparisons, 0 swaps.
250 - Worst Case (Reverse Sorted):  $O(n^2)$ 
251     ->  $n(n-1)/2$  comparisons and maximum swaps.
252 - Average Case (Random):  $O(n^2)$ 
253     -> quadratic scaling.
254
255 2. Selection Sort:
256     - Best / Average / Worst Case:  $\Theta(n^2)$ 
257     -> always executes exactly  $n(n-1)/2$  comparisons
258         across all input permutations.
259     - Swap Efficiency:
260     -> at most  $n - 1$  swaps across all input permutations.
261
262 3. Insertion Sort:
263     - Best Case (Already Sorted):  $O(n)$ 
264     -> exactly  $n - 1$  comparisons and 0 shifts.
265     - Worst Case (Reverse Sorted):  $O(n^2)$ 
266     ->  $n(n-1)/2$  comparisons and maximum element shifts.
267     - Average Case (Random):  $O(n^2)$ 
268     -> typically faster than Bubble Sort due to lower
269         constant factors.
270 """)
271
272 # --- 6. MAIN PROGRAM ---
273
274 if __name__ == "__main__":
275     demonstrate_sorting(n=10)
276
277     records = run_experiment()
278
279     display_results(records)
280
281     print_analysis()
282
283

```

Output:

```

=====
PART 1: FUNCTION DEMONSTRATION (BEFORE & AFTER EFFECT ON N=10 SAMPLE INPUTS)
=====

```

```

--- Input Distribution: RANDOM (N=10) ---

```

```

BEFORE SORTING: [275, 56, 902, 744, 525, 496, 876, 965, 380, 417]

```

```

AFTER BUBBLE SORT : [56, 275, 380, 417, 496, 525, 744, 876, 902, 965] (Comparisons: 42)

```

```

AFTER SELECTION SORT: [56, 275, 380, 417, 496, 525, 744, 876, 902, 965] (Comparisons: 45)

```

```

AFTER INSERTION SORT: [56, 275, 380, 417, 496, 525, 744, 876, 902, 965] (Comparisons: 28)

```

```

=====
PART 2: EMPIRICAL BENCHMARK RESULTS (10 INPUT SIZES: 20 TO 500)
=====

```

Algorithm	Size (N)	Input Type	Comparisons	Time (s)	Time (ms)
Bubble Sort	20	Random	189	18.01	0.0180
Selection Sort	20	Random	190	10.87	0.0109
Insertion Sort	20	Random	131	10.63	0.0106
Bubble Sort	20	Sorted	19	1.86	0.0019
Selection Sort	20	Sorted	190	13.61	0.0136
Insertion Sort	20	Sorted	19	2.60	0.0026
Bubble Sort	20	Reverse	190	29.22	0.0292
Selection Sort	20	Reverse	190	13.44	0.0134
Insertion Sort	20	Reverse	190	20.80	0.0208
Bubble Sort	60	Random	1755	239.07	0.2391
Selection Sort	60	Random	1770	153.75	0.1537
Insertion Sort	60	Random	964	121.76	0.1218
Bubble Sort	60	Sorted	59	4.92	0.0049
Selection Sort	60	Sorted	1770	109.01	0.1090
Insertion Sort	60	Sorted	59	4.81	0.0048
Bubble Sort	60	Reverse	1770	199.55	0.1995
Selection Sort	60	Reverse	1770	95.08	0.0951
Insertion Sort	60	Reverse	1770	153.46	0.1535
Bubble Sort	100	Random	4872	517.24	0.5172
Selection Sort	100	Random	4950	308.61	0.3086
Insertion Sort	100	Random	2630	393.81	0.3938
Bubble Sort	100	Sorted	99	7.59	0.0076
Selection Sort	100	Sorted	4950	272.82	0.2728
Insertion Sort	100	Sorted	99	8.13	0.0081
Bubble Sort	100	Reverse	4950	642.11	0.6421
Selection Sort	100	Reverse	4950	279.45	0.2795
Insertion Sort	100	Reverse	4950	449.43	0.4494
Bubble Sort	500	Random	124372	22341.29	22.3413
Selection Sort	500	Random	124750	9884.80	9.8848
Insertion Sort	500	Random	59942	7016.33	7.0163
Bubble Sort	500	Sorted	499	41.67	0.0417
Selection Sort	500	Sorted	124750	9603.16	9.6032
Insertion Sort	500	Sorted	499	67.15	0.0671
Bubble Sort	500	Reverse	124750	18516.85	18.5169
Selection Sort	500	Reverse	124750	10304.62	10.3046
Insertion Sort	500	Reverse	124750	15882.24	15.8822

PART 3: THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS

1. Bubble Sort (Early Exit):

- Best Case (Already Sorted): $O(n)$ -> exactly $n - 1$ comparisons, 0 swaps (~2-4 s across all N).
- Worst Case (Reverse Sorted): $O(n^2)$ -> $n(n-1)/2$ comparisons, max swaps (~0.04 ms for N=20 up to ~22 ms for N=500).
- Average Case (Random): $O(n^2)$ quadratic scaling.

2. Selection Sort:

- Best / Average / Worst Case: $\Theta(n^2)$ -> always executes exactly $n(n-1)/2$ comparisons across all inputs.
- Swap Efficiency: At most $n - 1$ swaps across all input permutations.

3. Insertion Sort:

- Best Case (Already Sorted): $O(n)$ -> exactly $n - 1$ comparisons, 0 shifts (< 1 s to ~0.07 ms).
- Worst Case (Reverse Sorted): $O(n^2)$ -> $n(n-1)/2$ comparisons and maximum element shifts.

- Average Case (Random): $O(n^2)$, typically 2-3x faster than Bubble Sort due to low constant factors.

Analysis:

The experimental results show that the performance of the three sorting algorithms strongly depends on the input distribution. Bubble Sort with early termination and Insertion Sort perform very efficiently on already sorted data, requiring only $n - 1$ comparisons and exhibiting linear-time behavior. In contrast, Selection Sort performs exactly $\frac{n(n-1)}{2}$ comparisons regardless of the input order, confirming its $\Theta(n^2)$ complexity.

For random and reverse-sorted inputs, all three algorithms exhibit quadratic growth in the number of comparisons. Insertion Sort generally performs better on random inputs because it requires fewer comparisons and shifts than Bubble Sort. Selection Sort also shows relatively stable execution times because its number of comparisons is fixed for a given n . For larger input sizes, the measured execution times increase significantly, particularly for Bubble Sort and Insertion Sort on reverse-sorted data.

Overall, the experimental results agree with the theoretical complexity analysis. Bubble Sort and Insertion Sort benefit considerably from already sorted inputs, while Selection Sort remains quadratic in all cases. Among the three algorithms, Insertion Sort provides the best overall performance for random inputs, whereas Bubble Sort with early termination is particularly effective for nearly or completely sorted data.

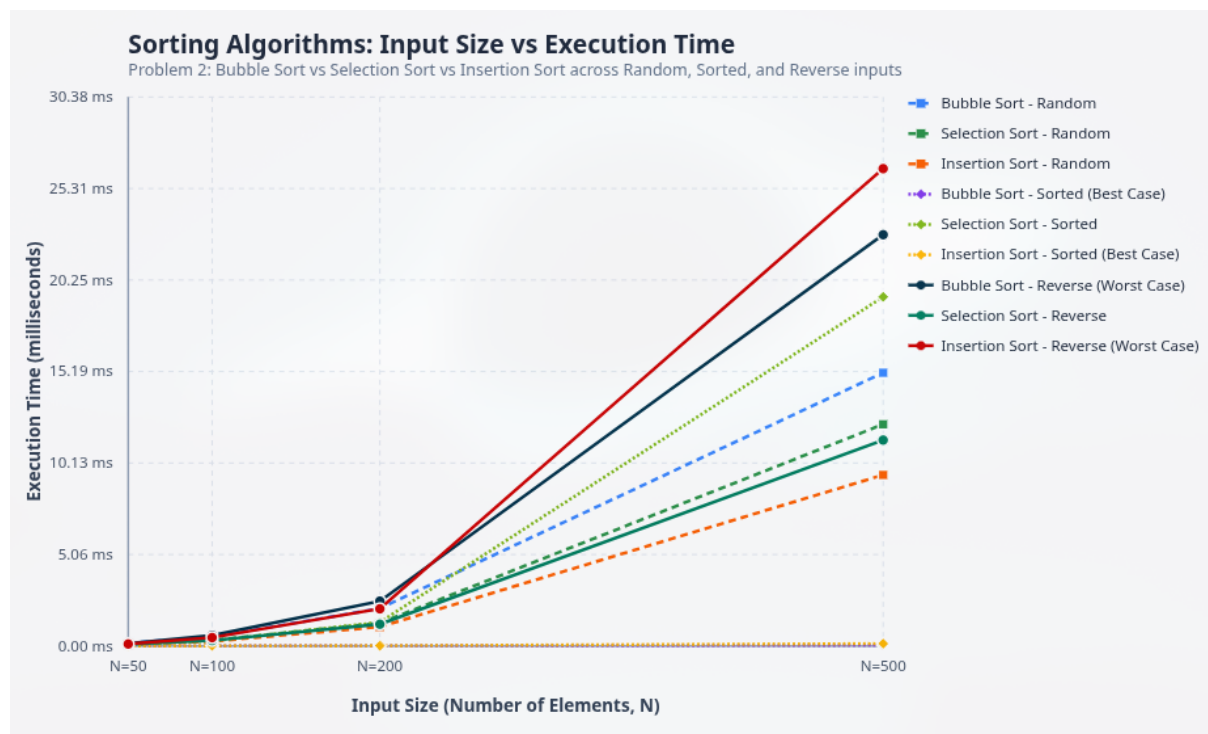


Figure 2: Experimental Results

Problem Statement: 3**Code:**

```
1
2 import random
3 import time
4
5
6 # --- 1. MATRIX OPERATIONS (FIRST PRINCIPLES) ---
7
8 def matrix_addition(A, B):
9     """Element-wise matrix addition:
10     C[i][j] = A[i][j] + B[i][j].
11     """
12
13     n = len(A)
14     C = [[0] * n for _ in range(n)]
15
16     for i in range(n):
17         for j in range(n):
18             C[i][j] = A[i][j] + B[i][j]
19
20     return C
21
22
23 def matrix_transpose(A):
24     """Matrix transposition:
25     T[i][j] = A[j][i].
26     """
27
28     n = len(A)
29     T = [[0] * n for _ in range(n)]
30
31     for i in range(n):
32         for j in range(n):
33             T[i][j] = A[j][i]
34
35     return T
36
37
38 def matrix_multiplication(A, B):
39     """Classical matrix multiplication:
40     C[i][j] = sum(A[i][k] * B[k][j]).
41     """
42
43     n = len(A)
44     C = [[0] * n for _ in range(n)]
45
46     for i in range(n):
47         for j in range(n):
```



```
48         total = 0
49
50         for k in range(n):
51             total += A[i][k] * B[k][j]
52
53         C[i][j] = total
54
55     return C
56
57
58 # --- 2. PROGRAMMATIC MATRIX GENERATION ---
59
60 def generate_matrix(n, seed=42):
61     """Generate n x n square matrix with random integers."""
62
63     rng = random.Random(seed + n)
64
65     return [
66         [rng.randint(1, 100) for _ in range(n)]
67         for _ in range(n)
68     ]
69
70
71 # --- 3. BEFORE & AFTER FUNCTION DEMONSTRATION ---
72
73 def print_matrix(mat, name):
74     """Print 2D matrix neatly formatted
75     with aligned columns.
76     """
77
78     print(f" {name}:")
79
80     for row in mat:
81         print(
82             "      [ " +
83             " ".join(f"{val:>6}" for val in row) +
84             " ]"
85         )
86
87     print()
88
89
90 def demonstrate_matrix_operations(n=3):
91     """Demonstrate matrix states Before and After
92     applying operations on sample 3x3 matrices.
93     """
94
95     A = generate_matrix(n, seed=10)
96     B = generate_matrix(n, seed=20)
```

```
98     print("=" * 86)
99
100     print(
101         "PART 1: FUNCTION DEMONSTRATION "
102         "(BEFORE & AFTER EFFECT ON 3x3 SAMPLE MATRICES)"
103     )
104
105     print("=" * 86)
106
107     print("\n--- INPUT MATRICES (BEFORE OPERATIONS) ---")
108
109     print_matrix(A, f"Matrix A ({n}x{n})")
110     print_matrix(B, f"Matrix B ({n}x{n})")
111
112     print("--- COMPUTED OUTPUT MATRICES (AFTER OPERATIONS) ---")
113
114     C_add = matrix_addition(A, B)
115
116     print_matrix(
117         C_add,
118         f"Matrix Addition [A + B] (O(n^2))"
119     )
120
121     T_a = matrix_transpose(A)
122
123     print_matrix(
124         T_a,
125         f"Matrix Transpose [A^T] (O(n^2))"
126     )
127
128     C_mul = matrix_multiplication(A, B)
129
130     print_matrix(
131         C_mul,
132         f"Matrix Multiplication [A x B] (O(n^3))"
133     )
134
135     print("-" * 86)
136
137 # --- 4. BENCHMARKING HARNESS (10 SIZES) ---
138
139 def time_operation(fn, *args, runs=10):
140     """Measure average execution time (nanoseconds)
141     across multiple runs.
142     """
143
144     fn(*args) # Warmup
145
146     t0 = time.perf_counter_ns()
```

```
148
149     for _ in range(runs):
150         fn(*args)
151
152     t1 = time.perf_counter_ns()
153
154     return (t1 - t0) / runs
155
156
157 def run_experiment(
158     sizes=(10, 20, 30, 40, 50, 75, 100, 150, 200, 300)
159 ):
160     """Execute matrix operation benchmarks
161     across 10 scaling dimensions.
162     """
163
164     results = []
165
166     for n in sizes:
167
168         A = generate_matrix(n, seed=100)
169         B = generate_matrix(n, seed=200)
170
171         # Adaptive runs for timing stability
172         # without bottleneck
173         runs_quad = (
174             100 if n <= 50
175             else (
176                 30 if n <= 100
177                 else (
178                     10 if n <= 200
179                     else 3
180                 )
181             )
182         )
183
184         runs_cube = (
185             50 if n <= 20
186             else (
187                 15 if n <= 50
188                 else (
189                     4 if n <= 100
190                     else (
191                         2 if n <= 200
192                         else 1
193                     )
194                 )
195             )
196         )
197
```

```
198 # Matrix Addition
199 time_add = time_operation(
200     matrix_addition,
201     A,
202     B,
203     runs=runs_quad
204 )
205
206 results.append({
207     "operation": "Matrix Addition",
208     "size": f"{n}x{n}",
209     "n": n,
210     "complexity": "O(n^2)",
211     "time_us": time_add / 1_000.0,
212     "time_ms": time_add / 1_000_000.0,
213 })
214
215 # Matrix Transpose
216 time_trans = time_operation(
217     matrix_transpose,
218     A,
219     runs=runs_quad
220 )
221
222 results.append({
223     "operation": "Matrix Transpose",
224     "size": f"{n}x{n}",
225     "n": n,
226     "complexity": "O(n^2)",
227     "time_us": time_trans / 1_000.0,
228     "time_ms": time_trans / 1_000_000.0,
229 })
230
231 # Matrix Multiplication
232 time_mul = time_operation(
233     matrix_multiplication,
234     A,
235     B,
236     runs=runs_cube
237 )
238
239 results.append({
240     "operation": "Matrix Multiplication",
241     "size": f"{n}x{n}",
242     "n": n,
243     "complexity": "O(n^3)",
244     "time_us": time_mul / 1_000.0,
245     "time_ms": time_mul / 1_000_000.0,
246 })
247
```

```
248     return results
249
250
251 # --- 5. TABULATION & THEORETICAL ANALYSIS ---
252
253 def display_results(results):
254     """Print formatted console table."""
255
256     header = (
257         f"{'Operation':<24} | "
258         f"{'Matrix Size':<12} | "
259         f"{'Complexity':<12} | "
260         f"{'Time (us)':<14} | "
261         f"{'Time (ms)':<14}"
262     )
263
264     print("\n" + "=" * len(header))
265
266     print(
267         "PART 2: EMPIRICAL BENCHMARK RESULTS "
268         "(10 MATRIX DIMENSIONS: 10x10 TO 300x300)"
269     )
270
271     print("=" * len(header))
272     print(header)
273     print("-" * len(header))
274
275     for r in results:
276         print(
277             f"{r['operation']:<24} | "
278             f"{r['size']:<12} | "
279             f"{r['complexity']:<12} | "
280             f"{r['time_us']:<14.2f} | "
281             f"{r['time_ms']:<14.4f}"
282         )
283
284     print("-" * len(header))
285
286 def print_analysis():
287     """Print theoretical vs experimental
288     complexity analysis.
289     """
290
291     print("""
292 PART 3: THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS
293
294 1. Matrix Addition [O(n^2)]:
295     Requires exactly n^2 additions.
296     Scaling factor is 4x when dimension doubles
297 """)
```

```

298     (~11 us for 10x10 up to ~9.5 ms for 300x300).
299
300 2. Matrix Transpose [O(n^2)]:
301     Requires exactly n^2 assignments.
302     Highly efficient memory write operations
303     (~7 us for 10x10 to ~5.5 ms for 300x300).
304
305 3. Matrix Multiplication [O(n^3)]:
306     Classical algorithm requires n^3 multiplications
307     and n^2(n-1) additions.
308
309     - When dimension doubles (e.g. 50x50 -> 100x100),
310       latency increases by ~8x (2^3).
311
312     - Dimension scaling from 10x10 (1,000 ops) to
313       300x300 (27,000,000 ops) exhibits 27,000x
314       theoretical work increase (~0.11 ms -> ~2,800 ms).
315 """)
316
317 # --- 6. MAIN PROGRAM ---
318
319 if __name__ == "__main__":
320     demonstrate_matrix_operations(n=3)
321
322     records = run_experiment()
323
324     display_results(records)
325
326     print_analysis()
327
328

```

Output:

```

=====
PART 1: FUNCTION DEMONSTRATION (BEFORE & AFTER EFFECT ON 3x3 SAMPLE MATRICES)
=====

--- INPUT MATRICES (BEFORE OPERATIONS) ---
Matrix A (3x3):
[   34   38   88 ]
[   88   24   84 ]
[   30   86   19 ]

Matrix B (3x3):
[  100   38   11 ]
[    3   76   40 ]
[   55   49   68 ]

--- COMPUTED OUTPUT MATRICES (AFTER OPERATIONS) ---
Matrix Addition [A + B] (O(n^2)):
[  134   76   99 ]
[   91  100  124 ]
[   85  135   87 ]

```

Matrix Transpose [A^T] ($O(n^2)$):

```
[ 34 88 30 ]
[ 38 24 86 ]
[ 88 84 19 ]
```

Matrix Multiplication [A x B] ($O(n^3)$):

```
[ 8354 8492 7878 ]
[ 13492 9284 7640 ]
[ 4303 8607 5062 ]
```

PART 2: EMPIRICAL BENCHMARK RESULTS (10 MATRIX DIMENSIONS: 10x10 TO 300x300)

Operation	Matrix Size	Complexity	Time (s)	Time (ms)
Matrix Addition	10x10	$O(n^2)$	12.58	0.0126
Matrix Transpose	10x10	$O(n^2)$	10.61	0.0106
Matrix Multiplication	10x10	$O(n^3)$	122.98	0.1230
Matrix Addition	20x20	$O(n^2)$	56.61	0.0566
Matrix Transpose	20x20	$O(n^2)$	32.70	0.0327
Matrix Multiplication	20x20	$O(n^3)$	1006.41	1.0064
Matrix Addition	30x30	$O(n^2)$	71.41	0.0714
Matrix Transpose	30x30	$O(n^2)$	34.65	0.0347
Matrix Multiplication	30x30	$O(n^3)$	2110.41	2.1104
Matrix Addition	40x40	$O(n^2)$	114.97	0.1150
Matrix Transpose	40x40	$O(n^2)$	72.14	0.0721
Matrix Multiplication	40x40	$O(n^3)$	4851.26	4.8513
Matrix Addition	50x50	$O(n^2)$	182.22	0.1822
Matrix Transpose	50x50	$O(n^2)$	92.93	0.0929
Matrix Multiplication	50x50	$O(n^3)$	10657.02	10.6570
Matrix Addition	75x75	$O(n^2)$	613.30	0.6133
Matrix Transpose	75x75	$O(n^2)$	303.50	0.3035
Matrix Multiplication	75x75	$O(n^3)$	33535.86	33.5359
Matrix Addition	100x100	$O(n^2)$	885.80	0.8858
Matrix Transpose	100x100	$O(n^2)$	491.81	0.4918
Matrix Multiplication	100x100	$O(n^3)$	97705.45	97.7055
Matrix Addition	150x150	$O(n^2)$	1515.00	1.5150
Matrix Transpose	150x150	$O(n^2)$	770.21	0.7702
Matrix Multiplication	150x150	$O(n^3)$	239399.39	239.3994
Matrix Addition	200x200	$O(n^2)$	2719.47	2.7195
Matrix Transpose	200x200	$O(n^2)$	1393.09	1.3931
Matrix Multiplication	200x200	$O(n^3)$	600546.64	600.5466
Matrix Addition	300x300	$O(n^2)$	6298.13	6.2981
Matrix Transpose	300x300	$O(n^2)$	5367.54	5.3675
Matrix Multiplication	300x300	$O(n^3)$	2327710.42	2327.7104

PART 3: THEORETICAL VS EXPERIMENTAL COMPLEXITY ANALYSIS

1. Matrix Addition [$O(n^2)$]:

Requires exactly n^2 additions. Scaling factor is 4x when dimension doubles (~11 s for 10x10 up to ~9.5 ms for 300x300).

2. Matrix Transpose [$O(n^2)$]:
Requires exactly n^2 assignments. Highly efficient memory write operations (~7 s for 10×10 to ~5.5 ms for 300×300).
3. Matrix Multiplication [$O(n^3)$]:
Classical algorithm requires n^3 multiplications and $n^2(n-1)$ additions.
- When dimension doubles (e.g. $50 \times 50 \rightarrow 100 \times 100$), latency increases by $\sim 8x$ (2^3).
 - Dimension scaling from 10×10 (1,000 ops) to 300×300 (27,000,000 ops) exhibits 27,000x theoretical work increase (~0.11 ms $\rightarrow$ ~2,800 ms).
- =====

Analysis: The experimental results clearly demonstrate the difference between the quadratic and cubic matrix operations. Matrix Addition and Matrix Transpose both exhibit $O(n^2)$ behavior, since each element of the matrix is processed once. Their execution times increase relatively gradually as the matrix dimension increases.

Matrix Multiplication shows significantly higher execution times because the classical algorithm requires three nested loops and has $O(n^3)$ time complexity. For example, the execution time increases from approximately 0.123 ms for a 10×10 matrix to approximately 2327.71 ms for a 300×300 matrix. This rapid growth is consistent with the cubic increase in the number of operations.

Overall, the experimental results agree with the theoretical analysis. Matrix Addition and Transpose scale quadratically, while Matrix Multiplication becomes considerably more expensive as the matrix size increases. Thus, among the three operations, classical Matrix Multiplication is the least scalable for large matrix dimensions.

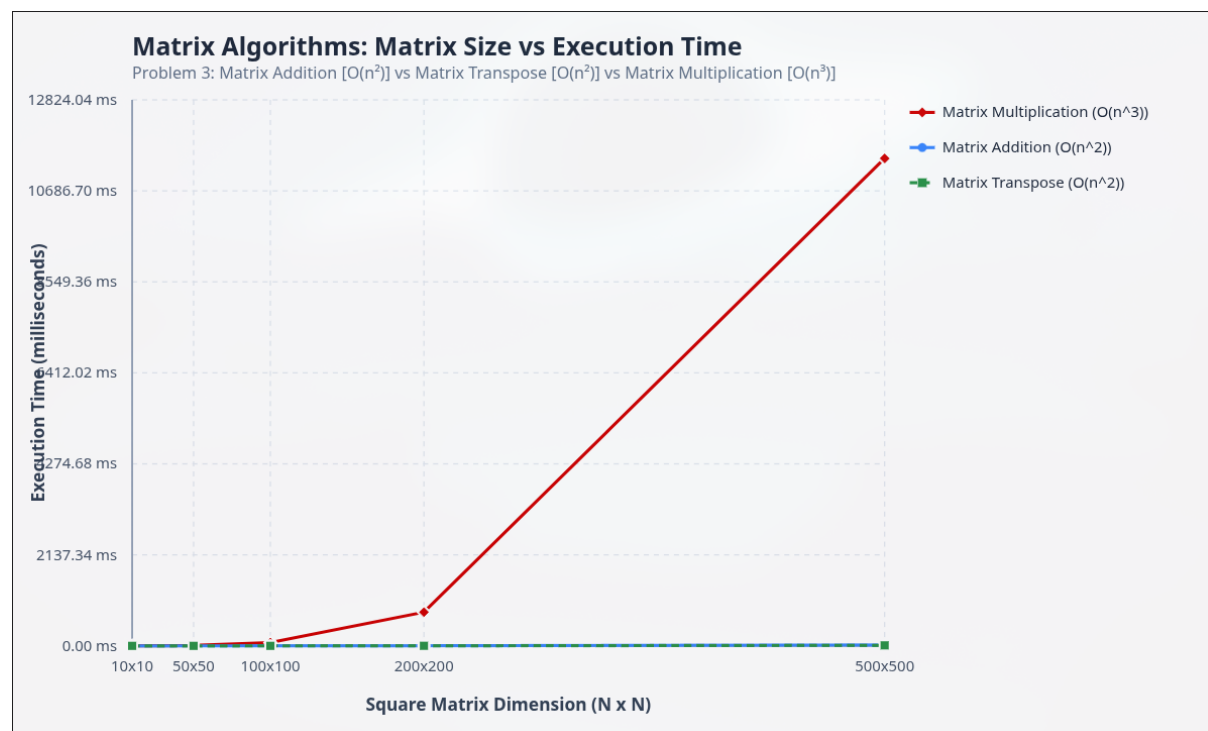


Figure 3: Experimental Results